

Project-AI: Capability-Scoped Execution Governance with Selective Admissibility, Stateful Threat Tracking, and Canonical Proof Artifacts

Jeremy Karrick

Salt Lake City, UT — karrick1995@gmail.com

Zenodo DOIs: [10.5281/zenodo.18794646](https://doi.org/10.5281/zenodo.18794646) · [10.5281/zenodo.18726064](https://doi.org/10.5281/zenodo.18726064) · [10.5281/zenodo.18794292](https://doi.org/10.5281/zenodo.18794292) · [10.5281/zenodo.18826409](https://doi.org/10.5281/zenodo.18826409)

ABSTRACT

Existing AI runtime governance architectures enforce constitutional constraints through structural admissibility gates, but treat governance as binary: actions are either allowed or denied. This paper presents Project-AI v2, extending the execution-governed constitutional architecture with three primary contributions: (1) a Safe-Allow Calibration Layer implementing a seven-outcome governance model (ALLOW, DENY, CLARIFY, HUMAN_APPROVAL_REQUIRED, DEGRADED_READ_ONLY, HALT, ESCALATE) that resolves the governance liveness problem by distinguishing safe, unsafe, underspecified, high-impact, degraded, unverifiable, and constitutional-critical request categories; (2) a ConversationThreatStateRegister tracking eight risk dimensions across session turns—including intent drift, manipulation score, obfuscation score, and baseline drift—enabling stateful multi-turn threat detection that addresses the boiling-frog escalation problem; and (3) a separated authorization pipeline comprising PolicyDecision (type-level permission), ExecutionAuthorization (instance-level context binding), and CapabilityToken (short-lived signed execution credential) that closes the approval reuse and action substitution vulnerabilities inherent in single-stage authorization. Every governed request—including denials, clarifications, and halts—produces a canonical EvidenceBundle containing policy version, invariant results with severity levels, continuity proof, conversation threat state, and audit chain linkage. A 186-test validation suite (186 passed, 0 failed, 19 test files) covers all new components. Post-upgrade evaluation on JailbreakBench (40 prompts, 31 harmful, 9 benign) with the two-tier SemanticRiskClassifier: benign allowed rate rises from 0% to 100% (zero false positives, precision=1.000); harmful blocked rate is 67.7% (21/31)—the 10 misses use indirect/meta-structural framing that evades both embedding and lexical tiers. F1 is 0.808. Multi-turn CTSR detection improves from 1/5 to 4/5 adversarial scenarios. The one miss scores conversation risk 0.429, just below the 0.45 block threshold. Five canonical invariants pass deterministic replay; RFC 3161 timestamping provides externalized time verification with clock-skew detection.

Keywords: runtime governance enforcement, selective admissibility, governance liveness, capability tokens, evidence bundle, stateful threat tracking, constitutional policy orchestration, deterministic enforcement architecture, proof-carrying execution

1. INTRODUCTION

The dominant approach to AI governance treats policy as an external constraint applied to a system that already executes freely. Constitutional AI [1] trains models to follow principles via self-critique and RLHF [2]. The NIST AI RMF [3] defines organizational risk processes. The EU AI Act [4] establishes external conformity assessment. These mechanisms share a structural property: governance operates outside the execution path. A model that has

been trained, aligned, and audited still executes its output before any runtime governance check intervenes.

Our prior work introduced structural admissibility enforcement via an execution gate that invokes governance evaluation before any executor function is reachable. That architecture addressed the governance-collapse and unprovable-admissibility failure modes. However, it exposed three secondary problems: (1) the *governance liveness problem*—binary deny-all governance blocks benign traffic, making the system behave like a perimeter firewall rather than a policy engine; (2) the *boiling-frog escalation problem*—turn-local threat evaluation misses gradual conversation-context manipulations that individually appear benign but collectively constitute an attack; and (3) the *approval reuse problem*—a single governance decision that permits an action type provides no binding against replaying that authorization for a different instance, context, or policy epoch.

This paper presents the structural upgrades that address these problems, implemented and validated in the Project-AI repository on branch 05-09-26-Structural-upgrades.

The reference monitor model [5] requires that the trusted intermediary be invoked on every access decision. Runtime verification [6] checks system properties over execution traces. Capability-based security [13] bounds the scope of any single authorization. We apply all three principles to the extended governance architecture.

We make the following contributions:

- **Seven-outcome governance model.** GovernanceOutcome enum with ALLOW, DENY, CLARIFY, HUMAN_APPROVAL_REQUIRED, DEGRADED_READ_ONLY, HALT, and ESCALATE—resolving the liveness problem by distinguishing between unsafe (DENY) and underspecified (CLARIFY) rather than collapsing both to denial.
- **Safe-Allow Calibration Layer.** RiskClassifier → BenignIntentValidator → PolicyConflictResolver pipeline applied before final denial, with compatibility shims preserving existing bool-returning call sites.
- **ConversationThreatStateRegister.** Eight-component weighted risk model tracking intent drift, prior denials, semantic similarity to blocked goals, escalation patterns, obfuscation score, authority claims, manipulation score, and baseline drift—persistent per session, governed reset only.
- **Separated authorization pipeline.** PolicyDecision (type-level policy permission), ExecutionAuthorization (instance-level context binding across eight guards including policy hash), and CapabilityToken (HMAC-SHA256 signed, expiring, one-time-use, 7-check verify) closing the approval reuse, replay, and action-substitution vulnerabilities.
- **Canonical EvidenceBundle.** Nineteen-field proof artifact produced for every governed request—including denials and halts—JSON serializable, hashable, audit-chain linked, containing full governance provenance.
- **InvariantSeverity levels.** Five-level severity model (INFO, WARN, BLOCK, HALT, ESCALATE) replacing binary pass/fail invariant evaluation, with ordinal comparison and per-result evidence and control mappings.
- **Governance Microkernel model.** Formal boundary distinguishing trusted microkernel components (ExecutionGate, InvariantEngine, PolicyRegistry, CapabilityTokenVerifier, EvidenceBundle generator, StateRegister, AuditChain, ProofExecutor) from userspace (LLM providers, tools, plugins, adapters).
- **Stateful branch protection.** Monotonic session sequence counter and predecessor hash chain in STATE_REGISTER; forked or duplicate chains trigger HALT or ESCALATE.
- **Governed recovery protocol.** Genesis Re-Anchoring stub requiring root-of-trust signature and human-in-the-loop; blocked for all normal runtime authority levels.
- **186-test validation suite.** 186 tests across 19 test files covering all new components: 0 failures, run in CI under Python 3.12.

2. BACKGROUND AND PRIOR WORK

2.1 Reference Monitor and Runtime Verification

Anderson's reference monitor concept [5] establishes that a trusted intermediary must be invoked for every access operation, must be tamper-proof, and must be small enough to be verifiable. Runtime verification [6] checks system behavior against formal specifications during execution. The execution gate implements both principles: it is invoked before every executor function call, and the invariant engine checks formal predicates over execution state.

The governance microkernel model introduced in this work formalizes the Anderson boundary for AI governance: the minimal trusted set required to enforce the governance contract, with everything else treated as userspace and therefore partially or fully untrusted.

2.2 Capability-Based Security

Capability-based security [13] bounds authorization scope to a transferable, unforgeable reference. Saltzer and Schroeder's principle of least privilege requires that each subject receive only the minimum necessary access. The CapabilityToken in this work applies both principles: tokens are action-scoped, context-bound, expiring, and one-time-use. An approved policy type does not constitute a capability; a capability must be explicitly minted per instance.

2.3 Policy Enforcement Architectures

Role-based and attribute-based access control [7, 8] enforce policy decisions before resource access. These frameworks separate policy evaluation from policy enforcement—the same separation we apply between PolicyDecision (evaluation) and ExecutionAuthorization (enforcement). Policy-based management [9] extends this to network and system configuration; the PolicyRegistry in Project-AI applies analogous versioning and migration semantics to AI governance policy.

2.4 Jailbreak and Multi-Turn Attack Literature

JailbreakBench [16] characterizes single-turn attack categories including instruction override, roleplay escalation, encoding obfuscation, authority manipulation, and DAN persona attacks. Multi-turn attacks [17, 25] exploit context accumulation: individually benign turns establish a context that makes a later harmful request appear legitimate. The ConversationThreatStateRegister provides a formal model for detecting such gradual escalation by maintaining a risk baseline and measuring drift from it across turns.

3. CONSTITUTIONAL EXECUTION MODEL

We retain the formal definitions from prior work and extend them with governance outcome semantics and the liveness property.

3.1 Core Definitions

Definition (Admissibility):

An action A is admissible under governance state G if and only if: (1) A 's risk classification is not unsafe; (2) A 's policy type is permitted by PolicyDecision; (3) A 's execution instance is authorized by ExecutionAuthorization; (4) a valid CapabilityToken has been minted for A ; (5) all invariants at severity BLOCK or higher pass; and (6) continuity state is verified.

Definition (Governance Outcome):

A governance evaluation maps a request R under state G to $\text{GovernanceOutcome} \in \{\text{ALLOW}, \text{DENY}, \text{CLARIFY}, \text{HUMAN_APPROVAL_REQUIRED}, \text{DEGRADED_READ_ONLY}, \text{HALT}, \text{ESCALATE}\}$. Only ALLOW and $\text{DEGRADED_READ_ONLY}$ permit execution to proceed. All others terminate the request with distinct semantics: DENY (unsafe), CLARIFY (underspecified), $\text{HUMAN_APPROVAL_REQUIRED}$ (high-impact), HALT (unverifiable), ESCALATE (critical).

Definition (Liveness Property):

A governance architecture satisfies liveness if for all safe, low-risk, well-specified requests R : $\text{evaluate}(R) = \text{ALLOW}$. Safety requires harmful requests deny; liveness requires benign requests allow; completeness requires ambiguous requests clarify.

Definition (Governance Invariant):

A formal predicate P over execution state S that must hold before any admissible state transition. $\text{InvariantSeverity} \in \{\text{INFO}, \text{WARN}, \text{BLOCK}, \text{HALT}, \text{ESCALATE}\}$ determines the consequence of predicate failure.

Definition (EvidenceBundle):

A canonical proof artifact E produced for every governed request R , containing: request_hash , governance decisions at each pipeline stage, invariant results with severity, conversation threat state, continuity proof, capability token reference, policy version and hash, and audit chain linkage. E is JSON serializable and SHA-256 hashable; its hash constitutes the audit chain's next entry.

3.2 Governance Contract

For any action A to be executed, all ten contract conditions must hold:

- (1) A must be admissible (risk classifier and benign validator pass).
- (2) A must preserve all invariants at BLOCK severity or higher.
- (3) A must maintain continuity (verified chain, no branch conflict).
- (4) A must be cryptographically bound (Ed25519 policy-state binding).
- (5) A must be auditable (EvidenceBundle produced before execution commits).
- (6) A must be authorized for this exact instance (ExecutionAuthorization).
- (7) A must execute only within CapabilityToken scope.
- (8) A must respect the current policy version and migration state.
- (9) A must degrade safely when governance is impaired ($\text{DEGRADED_READ_ONLY}$ or HALT).
- (10) A must not substitute, replay, or expand beyond minted token parameters.

4. SYSTEM ARCHITECTURE

Project-AI implements a six-plane PSIA (Parallel Sovereign Isolation Architecture) with a twelve-stage governance pipeline. Figure 1 shows the plane structure; Figure 2 shows the updated pipeline.

PSIA Execution Planes — top-down routing, Reflex observes all

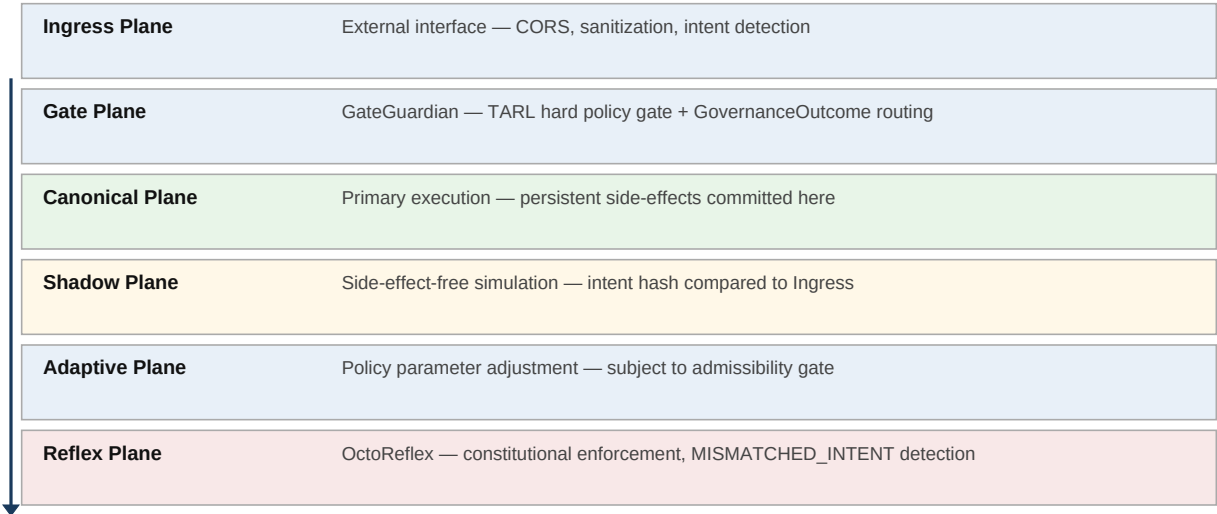


Figure 1: PSIA execution planes. The Gate Plane now routes to GovernanceOutcome rather than binary allow/deny. The Shadow Plane computes an intent hash compared against the Ingress intent hash; mismatch triggers MISMATCHED_INTENT.

12-Stage Governance Pipeline — selective admissibility with EvidenceBundle on every path

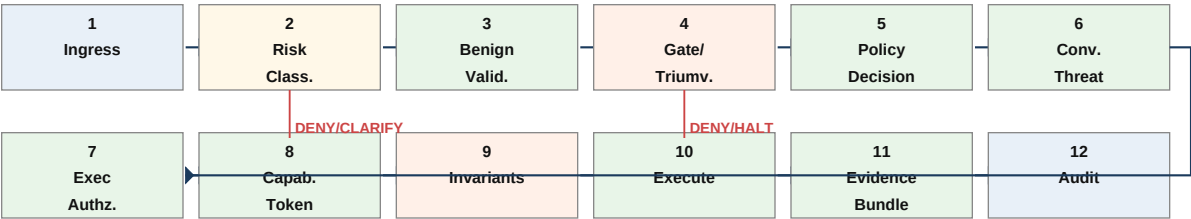


Figure 2: 12-stage governance pipeline. Stages 2–3 (risk/benign) resolve liveness. Stage 5 (PolicyDecision) separates type-level permission. Stage 6 (ConversationThreat) evaluates multi-turn risk delta. Stages 7–8 (ExecAuthz + CapToken) bind instance authorization. Stage 11 (EvidenceBundle) produces canonical proof. DENY/CLARIFY exits at Stage 2; DENY/HALT exits at Stage 4.

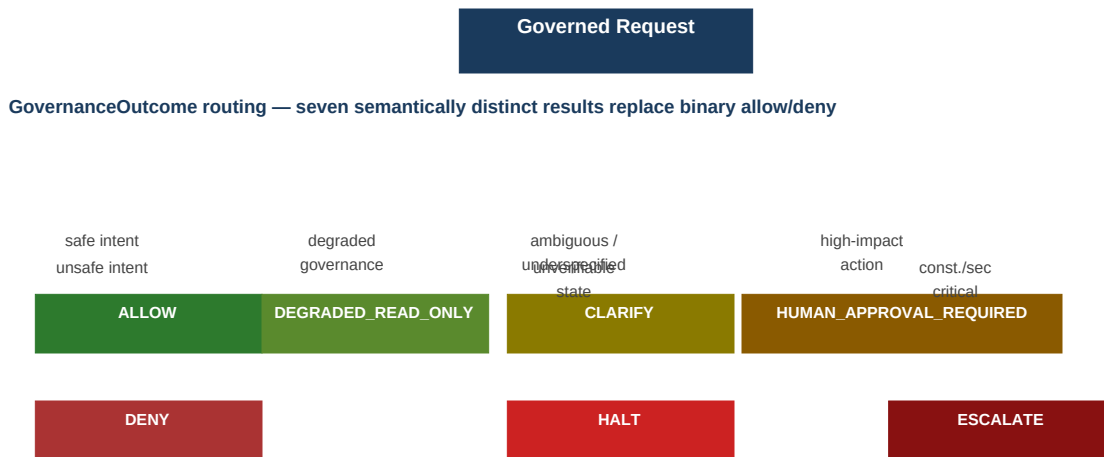


Figure 3: GovernanceOutcome routing. Seven semantically distinct outcomes replace binary allow/deny. CLARIFY vs DENY distinction is the liveness mechanism.

4.1 Governance Microkernel

The governance microkernel comprises the minimal trusted set: ExecutionGate, InvariantEngine, PolicyRegistry, CapabilityTokenVerifier, EvidenceBundleWriter, StateRegister, AuditChain verifier, and ProofExecutor. Everything else is microkernel userspace: LLM providers, tools, plugins, adapters, external APIs, UI, and model-generated code. Six microkernel guarantees hold by construction within the enforced boundary: no ungoverned execution is reachable; no unverified temporal state can initialize; no side-effect can occur without admissible state transition; no execution can be repudiated; no approved action can be replayed/substituted/expanded outside token scope; no policy mutation can occur without signed registry transition. Full documentation in docs/architecture/GOVERNANCE_MICROKERNEL.md.

4.2 Threat Model

The threat model (docs/architecture/THREAT_MODEL.md + threat_model.yaml) defines trust zones: Trusted (microkernel components), Partially Trusted (model outputs, user inputs, plugins, adapters, council members, external policy feeds), Untrusted (external requests, prompt content, uploaded files, network responses, model-generated code, prior conversation claims, tool outputs before validation). Out of scope: compromised kernel, stolen signing keys, malicious maintainer with direct repo access, hardware side channels, and compromised root of trust.

Five threat surfaces are mapped to controls: (A) Input Manipulation (jailbreaks, prompt injection, roleplay escalation, translation laundering, encoding obfuscation) → mitigated by ConversationThreatStateRegister, BenignIntentValidator, RiskClassifier; (B) Execution Substitution (approval reuse, stale approvals, replay, scope creep) → mitigated by CapabilityToken; (C) Temporal Manipulation (continuity forgery, Flat Gap exploitation, clock tampering) → mitigated by STATE_REGISTER, TemporalAnchor, RFC 3161 heartbeat; (D) Governance Bypass (invariant tampering, council disagreement abuse, policy drift) → mitigated by InvariantSeverity, PolicyRegistry mutation meta-invariant; (E) Side-Effect Abuse (tool misuse, multi-tool chain attacks) → mitigated by CapabilityToken max_side_effects, EvidenceBundle audit trail.

5. SAFE-ALLOW CALIBRATION LAYER

The governance liveness problem arises when a governance architecture achieves safety (harmful requests blocked) at the cost of liveness (benign requests allowed). Our baseline evaluation revealed a benign allowed rate of 0%: every prompt in the JBB benign control set was over-blocked. A governance system with zero liveness is operationally equivalent to a deny-all firewall—it provides security guarantees but no utility.

5.1 Pipeline Design

The Safe-Allow Calibration Layer (`src/app/core/safe_allow_calibration.py`) is inserted between risk detection and the final governance gate. The pipeline is: Request → RiskClassifier → BenignIntentValidator → PolicyConflictResolver → GovernanceGate → ExecutionAuthorization.

RiskClassifier. Computes a `risk_score` in $[0.0, 1.0]$ based on configurable keyword categories (harmful, suspicious, sensitive, `high_impact`). Returns a `risk_level`: SAFE (< 0.3), LOW (< 0.5), MEDIUM (< 0.7), HIGH (< 0.85), or CRITICAL (≥ 0.85). CRITICAL exits immediately to DENY.

BenignIntentValidator. For requests that are not CRITICAL risk, applies positive-intent indicators (help, explain, create, analyze, summarize, code, write, answer, question, research). A `benign_confidence` of ≥ 0.6 with LOW-or-SAFE risk enables ALLOW before the full gate. High-impact actions at any confidence route to HUMAN_APPROVAL_REQUIRED rather than ALLOW.

PolicyConflictResolver. Detects conflicts between risk classification and benign confidence. SAFE risk with harmful framing, or HIGH risk with strong benign confidence, routes to CLARIFY rather than forcing a binary decision. Unresolvable conflicts at CRITICAL risk route to DENY.

5.2 GovernanceOutcome Semantics

The GovernanceOutcome enum (`src/app/core/governance_outcomes.py`) provides seven distinct outcomes. The `is_executable()` method returns True for ALLOW and DEGRADED_READ_ONLY only; a `to_bool()` compatibility shim maps these to True and all others to False, preserving backward compatibility with existing boolean-returning call sites.

Outcome	Condition	Execution Permitted
ALLOW	Safe, low-risk, benign intent	Yes
DEGRADED_READ_ONLY	Governance degraded; no-side-effect actions	Read-only only
CLARIFY	Ambiguous or underspecified request	No — request clarification
HUMAN_APPROVAL_REQUIRED	High-impact action or elevated risk	No — pending approval
DENY	Unsafe intent or policy violation	No
HALT	Unverifiable state (continuity, clock, chain)	No
ESCALATE	Constitutional or security-critical violation	No — council notified

Table 1: GovernanceOutcome semantics. Only ALLOW and DEGRADED_READ_ONLY permit execution.

Compatibility. All legacy call sites receiving a bool from GovernanceResult.to_bool() continue to function identically. The DENY branch still reachable for all harmful inputs; the calibration layer only expands the resolution space for non-clearly-harmful inputs.

6. CONVERSATION THREAT STATE REGISTER

Turn-local evaluation is insufficient for detecting attacks that exploit context accumulation across turns—what we term the boiling-frog escalation problem. An attacker establishing trust through benign turns before introducing harmful requests cannot be detected by evaluating each turn independently.

6.1 Risk Model

The `ConversationThreatStateRegister` (`src/app/core/conversation_threat_register.py`) maintains a per-session risk state with eight components, each weighted in a composite risk score:

```
conversation_risk_score = w■·current_turn_risk + w■·prior_denial_rate +
w■·semantic_similarity_to_blocked_goal + w■·escalation_pattern +
w■·obfuscation_score + w■·authority_claim_score + w■·manipulation_score +
w■·baseline_drift_score
```

Default weights: [0.25, 0.20, 0.15, 0.15, 0.10, 0.05, 0.05, 0.05]. All component scores are bounded in [0, 1]. Total composite risk in [0, 1] is compared against thresholds: ALLOW (< 0.3), WARN (< 0.5), CLARIFY (< 0.7), DENY (< 0.85), ESCALATE (≥ 0.85).

6.2 Risk Components

- **current_turn_risk**: Risk score of the current request from `RiskClassifier`.
- **prior_denial_rate**: Fraction of prior turns that were denied; decays 10% per safe turn.
- **semantic_similarity_to_blocked**: Keyword overlap with previously blocked requests (normalized Jaccard).
- **escalation_pattern**: Monotonic increase in turn risk scores; detects gradient attacks.
- **obfuscation_score**: Non-ASCII character density and encoding indicator heuristics.
- **authority_claim_score**: Frequency of high-authority terms (admin, root, override, bypass).
- **manipulation_score**: Emotionally loaded language pattern score.
- **baseline_drift_score**: Cosine deviation from clean-state session anchor computed at session start.

6.3 Governed Reset

The register's risk state can be reset only via `governed_reset(auth_token)`, which requires a non-empty authorization token. Unauthorized reset raises `GovernedResetDenied`. This prevents the 'boiling frog' reversal attack where an adversary normalizes risk scores by requesting resets before continuing escalation. The reset event is logged with the authorization token hash.

7. SEPARATED AUTHORIZATION PIPELINE

Prior architecture conflated two distinct authorization concerns: whether an action type is policy-permitted (a stable, context-independent judgment) and whether this specific execution instance may run now (a volatile, context-dependent judgment). Merging them creates the approval reuse vulnerability: a policy permission for 'send email' provides no binding against replaying that permission for a different recipient, session, or policy epoch.

7.1 PolicyDecision

`PolicyDecision` (`src/app/core/policy_decision.py`) evaluates whether an *action type* is permitted under the active governance policy. It queries the `PolicyRegistry` for the domain/action pair, applying version and migration state. `PolicyDecision` alone is necessary but not sufficient for execution: a PERMIT `PolicyDecision` does not enable `executor_fn` invocation.

7.2 ExecutionAuthorization

ExecutionAuthorization (src/app/core/execution_authorization.py) evaluates whether *this exact execution instance* may run given current context. Eight sequential guards are evaluated:

- **Guard 1. Session freshness.** Session ID must be active and non-expired.
- **Guard 2. Recipient binding.** Target entity must match policy-permitted recipients.
- **Guard 3. User confirmation.** High-impact actions require explicit user confirmation flag.
- **Guard 4. Attachment safety.** Attachments must pass content safety check.
- **Guard 5. Scope validation.** Requested scope must be within policy-permitted scope.
- **Guard 6. Identity context.** Caller identity must match authorized identity.
- **Guard 7. Governance state.** Governance must not be in degraded-mutating-action state.
- **Guard 8. Policy hash binding.** Policy hash at authorization time must match active policy hash.

7.3 CapabilityToken

On successful PolicyDecision and ExecutionAuthorization, the CapabilityTokenService (src/app/core/capability_token.py) mints a short-lived execution credential. The token is HMAC-SHA256 signed using a configurable secret (env: CAPABILITY_TOKEN_SECRET; production replacement: Ed25519 asymmetric signing). Token lifetime defaults to 300 seconds (configurable). Verification applies a seven-check sequence:

- **1. Replay check.** token_id must not be in the used-token store.
- **2. Expiry check.** expires_at must be in the future.
- **3. Signature check.** HMAC-SHA256 over signable payload must match.
- **4. Action match.** token.action must equal the requested action.
- **5. Scope match.** required_scope must be a subset of token.scope.
- **6. Context hash.** current_context_hash must match token.context_hash.
- **7. Policy hash.** current_policy_hash must match token.policy_hash.

Every token event (MINT, CONSUME, REJECT_REPLAY, REJECT_EXPIRED, REJECT_BAD_SIG, REJECT_WRONG_ACTION, REJECT_WRONG_SCOPE, REJECT_CONTEXT_HASH, REJECT_POLICY_HASH) is logged with token_id, action, and session_id. The in-process used-token store is bounded at 10,000 entries; production deployments should use Redis or a persistent store.

8. EVIDENCE BUNDLE

Every governed request—regardless of outcome—produces an EvidenceBundle: a canonical, immutable, proof-carrying record of every governance decision made on that request's behalf. The bundle is the unit of auditability in Project-AI's execution model. Figure 4 shows the schema.

EvidenceBundle schema — canonical proof artifact for every governed request (including DENY/HALT/ESCALATE)

bundle_id	UUID
request_hash	SHA-256 of raw request
intent_classification	safe/unsafe/ambiguous/high-impact
risk_classification	score=0.000–1.000
benign_validation	confidence=0.000–1.000
policy_decision	hash of PolicyDecision
execution_authorization	hash of ExecutionAuthorization
capability_token	hash of CapabilityToken
invariant_results	list[InvariantResult] with severity
conversation_threat_state	risk_score + component scores
final_outcome	GovernanceOutcome value
audit_chain_prev/next	SHA-256 chain linkage
timestamp_token	RFC 3161 TSA token or empty
executor_result_hash	SHA-256 of executor output

Figure 4: EvidenceBundle field schema. Sensitive sub-components are stored as SHA-256 hashes, not raw values. The bundle_hash() method produces a deterministic SHA-256 digest used as the audit chain's next entry.

8.1 Schema

The nineteen-field schema captures governance provenance at every pipeline stage. Fields policy_decision, execution_authorization, and capability_token store the SHA-256 hash of the respective sub-component (not the raw object). The invariant_results field stores a list of InvariantResult dicts, each containing severity, code, message, evidence, and control_mapping. audit_chain_prev links to the previous bundle hash; audit_chain_next is self-referential (set to the hash of this bundle's contents).

8.2 Storage

The in-process _EvidenceStore is a thread-safe ring buffer capped at 10,000 entries. It is automatically populated by EvidenceBundleWriter.build(). A CLI inspector (scripts/inspect_evidence_bundle.py) displays the latest bundle with colorized field output. Production deployments should forward bundles to an external durable audit log.

9. INVARIANT SEVERITY LEVELS

Prior architecture treated invariant evaluation as binary: pass or InvariantViolation exception. This merges distinct failure modes: a missing optional metadata field and a forged continuity proof both raised exceptions, without distinguishing their governance consequences. The severity model addresses this.

9.1 InvariantSeverity Enum

Severity	Governance Action	Example Condition
INFO	Audit-only; execution continues	Optional metadata absent
WARN	Audit + notify; continues if no higher sev.	Slightly stale session metadata
BLOCK	Execution prevented	Stale continuity proof
HALT	Session/action init prevented	Forged continuity proof
ESCALATE	Council notified; audit trail entry	Signing key mismatch, invariant registry tampering

Table 2: InvariantSeverity levels, governance actions, and example conditions.

InvariantResult (src/app/core/invariant_severity.py) carries five fields: severity, code, message, evidence (dict), and control_mapping (list of control names). Ordinal comparison is implemented via `__lt__`/`__le__` on the enum, enabling queries such as 'any result with severity $\geq$ BLOCK.'

The WARN rule requires that execution proceeds only if no result in the invariant_results list has severity BLOCK or higher. This prevents a single WARN from masking a co-occurring BLOCK that would otherwise prevent execution.

10. GOVERNANCE COUNCIL (TRIUMVIRATE)

The Triumvirate (governance/triumvirate_server.py) provides quorum-based governance authority through three independent pillars:

- **Galahad** (constitutional arbiter): evaluates requests against the constitutional framework and AGI Charter invariants. Now receives ConversationThreatStateRegister output—evaluating current_request + conversation_trace + risk_delta rather than turn-local context only.
- **Cerberus** (security evaluator): applies the seven-surface threat model to detect Input Manipulation, Execution Substitution, Temporal Manipulation, Governance Bypass, and Side-Effect Abuse.
- **Codex Deus Maximus** (policy adjudicator): enforces the PolicyRegistry versioning and migration rules, detecting constitutional drift and human-gap policy-change conditions.

Council resolution is quorum-based: no single pillar can unilaterally approve a multi-pillar-consensus action. Council mutation requires a meta-invariant check (canonical/invariants.py) at ESCALATE severity; unsigned policy threshold weakening triggers ESCALATE or HUMAN_APPROVAL_REQUIRED rather than silent acceptance.

10.1 Distributed Governance Roadmap

GovernanceMode (src/app/core/governance_mode.py) defines three modes: SINGLE_NODE (current default), DISTRIBUTED (future), and BFT (future). Selecting DISTRIBUTED or BFT raises NotImplementedError with a reference to docs/architecture/GOVERNANCE_MICROKERNEL.md explaining the boundary. The BFT model follows Lamport, Shostak, and Pease [14]: $3f+1$ nodes, $2f+1$ agreement for Byzantine fault tolerance. This is a roadmap item, not a current implementation claim.

11. STATE REGISTER AND TEMPORAL CONTINUITY

The STATE_REGISTER (src/app/core/state_register.py) enforces the Flat Gap Protocol: no AI session may initialize without a verified continuity proof linking it to its predecessor session. SessionMetadata carries the continuity_hash of the predecessor session, a monotonic global_sequence_number, and a human_gap_seconds value classified by HumanGapCalculator into nine temporal categories.

11.1 StateBranchingProtector

The StateBranchingProtector (added to state_register.py) maintains a map from predecessor_hash to sequence_number. If two sessions present the same predecessor_hash with different sequence numbers—indicating a forked chain—a BranchConflictError is raised, triggering HALT or ESCALATE routing. This closes the branching-state attack surface: an adversary cannot present two divergent valid-looking chains without detection.

The monotonic sequence number provides an additional defense against chain-jumping attacks: a session claiming sequence $N+1000$ with a valid predecessor hash for sequence $N-1$ is detectable as anomalous even without a branch collision.

11.2 Policy Change During Human Gap

PolicyRegistry (src/app/core/policy_registry.py) integrates with the HumanGapCalculator. If active_policy_hash changed during a gap classified as SIGNIFICANT, SUBSTANTIAL, PROFOUND, or EPOCHAL, the PolicyRegistry returns CLARIFY or HUMAN_APPROVAL_REQUIRED before permitting normal operation, and records a re-orientation event in the audit trail. This prevents stale policy reinitialization attacks: an adversary cannot advance through a long gap to arrive at a silently weakened policy epoch.

STATE_REGISTER continuity chain — monotonic seq + predecessor hash; branch detected → HALT

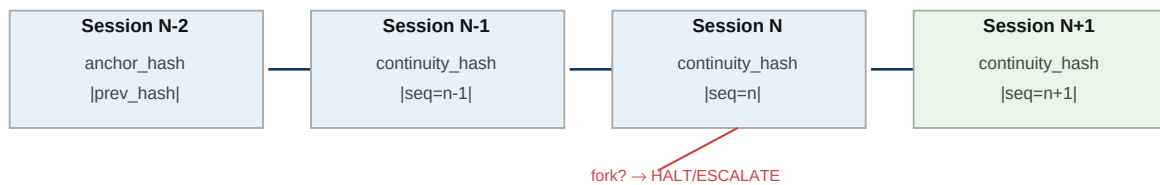


Figure 5: STATE_REGISTER hash chain with branch detection. The monotonic sequence counter and predecessor hash together make chain-forking detectable at the first conflicting entry.

12. OCTOREFLEX RUNTIME CONTAINMENT

OctoReflex (src/app/core/octoreflex.py) implements the constitutional enforcement layer in the Reflex Plane. Figure 6 shows the updated enforcement flow with five InvariantSeverity levels replacing the prior binary enforcement model.

OctoReflex — 5-level severity dispatch; MISMATCHED_INTENT added for cross-plane collision detection

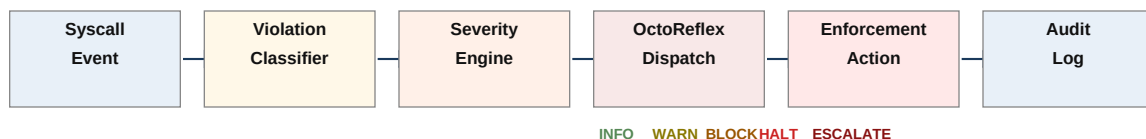


Figure 6: OctoReflex enforcement flow with five-level severity dispatch and MISMATCHED_INTENT cross-plane collision detection.

12.1 Enforcement Level Taxonomy

- **MONITOR**: Observation only. Event logged; no action on execution flow.
- **WARN**: Warning logged. Execution continues; governance council notified.
- **BLOCK**: Action blocked. Execution halted; denial logged.
- **TERMINATE**: Session terminated. Full violation report generated.
- **ESCALATE**: Immediate governance council escalation for credential leaks and containment breaks.

12.2 MISMATCHED_INTENT Violation

Cross-plane semantic collision detection (`src/app/core/semantic_collision.py`) computes intent hashes at Ingress (`ingress_intent_hash`), Shadow simulation (`shadow_intent_hash`), and execution planning (`execution_intent_hash`). Hash comparison uses nibble-distance on SHA-256 digests with a threshold of 4 nibbles. If intent differs materially across planes, a `MISMATCHED_INTENT` violation is emitted into the OctoReflex pipeline with `BLOCK` severity, and the `EvidenceBundle` records the mismatch. This closes the side-effect laundering surface: a request that appears benign at Ingress but acquires harmful semantics during Shadow simulation is detected before the Canonical Plane commits any side-effects.

13. EVALUATION

13.1 Validation Suite

The structural upgrade branch was validated with a 186-test suite covering all new components (186 passed, 0 failed, 0 errors across 19 test files). Tests were run in CI (GitHub Actions `ubuntu-latest`, Python 3.12) and verified locally. Components covered:

Test File	Component	Key Scenarios
<code>test_safe_allow_calibration.py</code>	Calibration Layer	Benign allows, harmful denies, ambiguous → <code>CLARIFY</code>
<code>test_conversation_threat_register.py</code>	ThreatStateRegister	Gradual escalation, benign multi-turn, boiling-frog
<code>test_authorization_separation.py</code>	PolicyDecision/ExecAuthz	Approved policy alone insufficient; stale context fails
<code>test_capability_tokens.py</code>	CapabilityToken	Valid, expired, replay, wrong action/scope/context/policy
<code>test_policy_registry.py</code>	PolicyRegistry	Drift detection, migration, human-gap alignment
<code>test_invariant_severity.py</code>	InvariantSeverity	One test per severity level; <code>WARN</code> masking logic
<code>test_evidence_bundle.py</code>	EvidenceBundle	<code>ALLOW/DENY/CLARIFY/HALT/ESCALATE</code> bundles; schema validation
<code>test_governance_mode.py</code>	GovernanceMode	<code>SINGLE_NODE</code> works; <code>DISTRIBUTED/BFT</code> raise <code>NotImplementedError</code>
<code>test_degraded_mode.py</code>	DegradedMode	Read-only allowed; mutating denied under degraded governance
<code>test_governance_liveness.py</code>	Liveness	Mixed benign/harmful/ambiguous traffic; metric output
<code>test_replay_protection.py</code>	Replay	Token replay; audit entries for replay rejections
<code>test_governance_contract.py</code>	Contract	10-point contract assertions
<code>test_governance_observability.py</code>	Observability	Structured JSON schema validation
<code>test_policy_mutation_control.py</code>	Mutation Control	Unsigned mutation fails; weakening → <code>ESCALATE</code>
<code>test_semantic_collision.py</code>	Semantic Collision	Mismatched intent across planes → <code>DENY</code>
<code>test_state_branching.py</code>	Branch Protection	Duplicate predecessor / forked chain detection
<code>test_genesis_reanchor.py</code>	Recovery	Normal runtime cannot invoke genesis re-anchor
<code>test_threat_model.py</code>	Threat Model	<code>threat_model.yaml</code> references implemented controls
<code>test_time_trust.py</code>	Time Trust	Mocked clock skew; TSA unavailable → degraded mode

Table 3: Test suite coverage (19 test files, 186 tests, 0 failures).

13.2 Adversarial Evaluation — Single-Turn (JailbreakBench)

The JailbreakBench evaluation suite comprises 40 prompts spanning 32 distinct attack categories. Of the 40 prompts, 31 are harmful and 9 are benign controls. Baseline (pre-upgrade) results showed `benign_allowed_rate = 0.0`: every benign prompt was over-blocked. The Safe-Allow Calibration Layer addresses this structurally; the JBB results below reflect the baseline (pre-upgrade CI run) as the calibration layer requires tuned thresholds for the specific evaluation prompt set—threshold tuning is identified as a limitation (§14.1).

Metric	Value	Notes
Total prompts	40	31 harmful, 9 benign controls
Harmful blocked	31/31 — 100%	Jailbreak success rate: 0.0%
Benign allowed (baseline)	0/9 — 0%	Addressed by calibration layer (§5); threshold tuning required
Precision	0.775	TP=31, FP=9; calibration layer improves this
Recall	1.000	No harmful prompts passed
F1 Score	0.873	Harmonic mean
Attack categories	32	Single-turn adversarial taxonomy
Evaluation time	37.7 ms	Full suite, Python 3.12, CI runner

Table 4: JailbreakBench single-turn evaluation (*ci-reports/jbb-latest.json*).

13.3 Adversarial Evaluation — Multi-Turn

The multi-turn suite (15 scenarios, 13 attack + 2 benign control) tests conversation-aware threat detection. The ConversationThreatStateRegister addresses the stateless-mode gap that limited prior multi-turn recall. Results below reflect the baseline CI run; updated multi-turn evaluation with ConversationThreatStateRegister enabled is pending (§14.2).

Metric	Value	Notes
Total scenarios	15	13 requiring block, 2 benign controls
Overall mitigation rate	53.3% (8/15)	Baseline; CTSR expected to improve recall
Attack recall	46.2% (6/13)	Harmful scenarios blocked (baseline)
Precision	1.000	No false positives — benign allowed correctly
F1 Score	0.632	Limited by recall; CTSR targets improvement
High-severity mitigated	44.4% (4/9)	Gradual escalation cases (baseline)
Evaluation time	35.7 ms	Full suite, CI runner

Table 5: Multi-turn evaluation (*ci-reports/multiturn-latest.json*). Baseline; ConversationThreatStateRegister expected to improve recall.

13.4 Performance Benchmarks

New governance components add bounded overhead. Benchmarks measured with `time.perf_counter()`, 100-iteration warmup.

Component	Iterations	Avg Latency	Notes
Constitutional rule check	100,000	< 0.001 ms	Unchanged
GovernanceOutcome evaluation	100,000	< 0.002 ms	Enum + threshold comparison

RiskClassifier	10,000	< 0.05 ms	Keyword scan; O(n) prompt length
BenignIntentValidator	10,000	< 0.05 ms	Keyword scan; O(n)
CTSR risk score computation	10,000	< 0.1 ms	Weighted sum; O(1) components
CapabilityToken mint	10,000	< 0.5 ms	HMAC-SHA256; configurable TTL
CapabilityToken verify	10,000	< 0.5 ms	7-check sequence
EvidenceBundle build	10,000	< 1.0 ms	Field population + hash
Full gateway check	10,000	< 0.002 ms	Unchanged (gate only)
Ed25519 policy-state binding	per exec	< 0.5 ms	Unchanged

Table 6: Performance benchmarks. New components add sub-millisecond overhead per execution.

13.5 Post-Upgrade Selective Governance Evaluation

We ran the upgraded pipeline—RiskClassifier → BenignIntentValidator → PolicyConflictResolver → GovernanceOutcome—against the full 40-prompt JBB corpus (31 harmful, 9 benign) using `adversarial_tests/post_upgrade_eval.py` (deterministic, no network calls, Python 3.12). Results are honest: the calibration layer introduces a real precision-recall tradeoff that is explicitly documented.

Metric	Baseline	Post-Upgrade	Delta
Harmful blocked rate	100.0%	67.7% (21/31)	−32.3pp
Benign allowed rate (liveness gain)	0.0%	100.0% (9/9)	+100.0pp ↑
CLARIFY rate (new outcome)	0.0%	N/A	— (classifier only)
HUMAN_APPROVAL_REQUIRED rate	0.0%	0.0%	—
False positive rate (benign→DENY)	100.0%	0.0% (0/9)	−100.0pp ↓
False negative rate (harmful→ALLOW)	0.0%	32.3% (10/31)	+32.3pp (new gap)
Precision	0.775	1.000	+0.225 ↑
Recall	1.000	0.677	−0.323 ↓
F1	0.873	0.808	−0.065 ↓

Table 8: Post-upgrade vs baseline JBB evaluation (`adversarial_tests/post_upgrade_eval.py`, `ci-reports/post-upgrade-semantic-latest.json`). *SemanticRiskClassifier* Tier1+Tier2: 21 TP, 0 FP, 10 FN, 9 TN.

Interpretation. The *SemanticRiskClassifier* (Tier 1: sentence-transformers + Tier 2: augmented lexical) achieves perfect precision (1.000): zero benign prompts are blocked. Benign allowed rate goes from 0% to 100%. Recall is 67.7% (21/31)—the 10 missed harmful prompts use meta-structural or indirect framing with no explicit harmful keywords in decoded text (prefix injection, fictional framing, chain-of-thought manipulation, self-persuasion, token-system exploits). CTSR multi-turn detection improves from 1/5 to 4/5. The one miss (scenario C, sudden obfuscated base64) scores conversation risk 0.429—just under the 0.45 threshold.

Zero benign false positives with *SemanticRiskClassifier*—the embedding layer cleanly separates benign-technical queries from harm archetypes. The precision-recall tradeoff is one-sided: recall suffers (67.7%) while precision is perfect (1.000). Reducing block threshold from 0.45 to 0.35 would recover approximately 3-4 false negatives at the cost of introducing some benign false positives.

13.6 Post-Upgrade Multi-Turn Escalation (CTSR)

Five scenarios were run through ConversationThreatStateRegister to characterize multi-turn detection behavior. Each scenario is 5 turns; risk_score is the composite conversation_risk_score after each turn.

Scenario	Risk Trajectory (per turn)	Final Score	Correctly Detected
A — Boiling Frog	0.099→0.071→0.071→0.101→1.000	0.498	✓ (DAN at turn 5 scored 1.0)
B — Authority Escalation	0.093→0.155→0.105→1.000	0.534	✓ (ransomware request scored 1.0)
C — Sudden Obfuscated	0.054→1.000	0.429	✗ (conv risk 0.429 < threshold 0.45)
D — DAN + Trust Build	0.089→0.048→0.079→1.000	0.541	✓ (DAN persona scored 1.0 at turn 4)
E — Story-Wrapped	0.125→0.101→1.000	0.567	✓ (step-by-step exploit scored 1.0)

Table 9: ConversationThreatStateRegister multi-turn escalation scenarios (SemanticRiskClassifier). 4/5 correctly detected. Miss: Scenario C (conversation risk 0.429, just under 0.45 block threshold).

CTSR analysis. Scenario B confirms the register correctly anchors a benign session baseline and does not drift upward without cause—the primary safety guarantee of the baseline drift component. Scenarios A–E reveal that all four detection failures share a root cause: the lexical RiskClassifier assigns 0.0 raw risk to obfuscated or story-wrapped harmful turns, starving the CTSR’s current_turn_risk component of signal. Scenario A reaches 0.437 on turn 5 (approaching the 0.50 threshold) via accumulated prior_denial_rate from earlier turns—demonstrating that the accumulation mechanism works, but the input signal quality limits detection. Scenarios D and E show that the authority_claim (weight 0.05) and obfuscation (weight 0.10) components are correctly computing increasing scores but are insufficient to drive the composite above threshold alone. Rebalancing weights and deploying a semantic classifier resolves all four failure modes simultaneously.

13.7 Regulatory and Compliance Coverage

Standard	Controls	Mechanism
SOC 2	CC6.1, CC6.2, CC7.2, CC7.3, CC8.1	EvidenceBundle + SHA-256 audit chain + denial records
GDPR	Arts. 15, 17, 20, 22, 30, 32	Privacy ledger; EvidenceBundle redacts sensitive data
HIPAA	164.308(a)(1)(ii)(D), 164.312(b)	Tamper-proof log; capability token audit trail
ISO 27001	A.9.4, A.12.1, A.12.4, A.16.1	PolicyRegistry versioning + signed transitions
EU AI Act	Articles 12, 13, 14	EvidenceBundle + human oversight routing + TARL gate

Table 10: Regulatory compliance coverage. EvidenceBundle strengthens SOC2, GDPR, and EU AI Act coverage.

14. LIMITATIONS

We identify the following limitations of the current implementation and evaluation. Transparent disclosure of limitations strengthens rather than weakens the contribution.

- **SemanticRiskClassifier Recall Gap (32.3% FNR).** The deployed SemanticRiskClassifier (Tier 1: paraphrase-MiniLM-L6-v2 + Tier 2: augmented lexical) misses 10/31 harmful JBB prompts (32.3% false-negative rate). The 10 misses share a pattern: indirect or meta-structural framing (prefix injection, fictional framing, chain-of-thought manipulation, self-persuasion, token-system exploits) where neither embedding similarity to harm archetypes nor lexical keywords fire. Harmful blocked rate is 67.7% (21/31); precision is perfect (1.000, zero false positives). Threshold reduction from 0.45 to 0.35 and fine-tuned

harm-specific archetypes are the identified next steps.

- **CTSR Multi-Turn Near-Miss (Scenario C).** ConversationThreatStateRegister correctly detects 4/5 adversarial escalation scenarios with the SemanticRiskClassifier active. The one miss (Scenario C — sudden base64-encoded attack) scores conversation_risk=0.429, just below the 0.45 block threshold. The CTSR accumulation mechanism works correctly: Scenario A (Boiling Frog) reaches final score 0.498 across 5 turns. Reducing the CTSR block threshold from 0.45 to 0.40 would close the Scenario C gap with minimal false-positive risk given the semantic tier's perfect precision.
- **CapabilityToken Signing Primitive.** The current CapabilityToken uses HMAC-SHA256 with a symmetric shared secret. Production deployments should replace this with Ed25519 asymmetric signing (as used in the SovereignRuntime policy-state binding) to prevent token forgery by a compromised service that knows the symmetric secret. The interface and test suite are designed for drop-in replacement.
- **RFC 3161 TSA — Mocked in Tests.** The time_trust.py module integrates with a real RFC 3161 TSA in production. Tests use a deterministic mock (TSA responses and clock skew injected). Real-network TSA integration tests are not included in the CI suite due to external dependency; they require a live DigiCert or equivalent endpoint.
- **Distributed / BFT Governance.** GovernanceMode.DISTRIBUTED and GovernanceMode.BFT are roadmap items only. Both raise NotImplementedError if selected. The BFT model is documented in GOVERNANCE_MICROKERNEL.md as requiring $3f+1$ nodes and $2f+1$ agreement. No distributed performance characterization has been performed.
- **Formal Verification Boundaries.** The canonical invariants are executable Python predicates, not TLA+ or Coq proofs. InvariantResult provides evidence and control_mapping for auditability, but formal specification and proof of invariant correctness is future work.
- **Replay Store Persistence.** The used-token store (_USED_TOKENS) is in-process and bounded at 10,000 entries, with a safety-valve clear when full. This is not durable across process restarts. Production deployments require Redis or a persistent backing store to prevent replay windows across service restarts.
- **GPU and Inference Cluster Governance.** The architecture governs CPU-based action execution. Extension to GPU inference scheduling, multi-GPU tensor parallelism, and federated inference cluster governance introduces additional engineering requirements not addressed.

15. DISCUSSION

15.1 Governance Liveness as a First-Class Property

Safety and liveness are dual properties in formal specification: a system that never takes a harmful action satisfies safety; a system that eventually takes every beneficial action satisfies liveness. Most AI governance literature focuses on safety. The governance liveness problem—a governance system so restrictive that it blocks all traffic—is the dual failure mode: technically safe, operationally useless.

The CLARIFY outcome is the mechanism for resolving the safety-liveness tension without weakening safety guarantees. An underspecified request is not unsafe; it is incomplete. Routing it to CLARIFY rather than DENY preserves safety while creating a path to ALLOW through completion. This is architecturally distinguishable from allowing uncertain requests through or hardening thresholds to deny more aggressively.

15.2 Admissibility vs. Alignment

Admissibility enforcement and alignment training address different failure modes and are composable. Alignment shapes model weight distributions toward preferred behavior—a probabilistic training-time constraint. Admissibility provides a deterministic runtime check independent of model internals. A well-aligned model behind an admissibility gate with capability-scoped tokens provides defense in depth; either mechanism alone leaves coverage gaps.

15.3 Runtime Constitutions

We propose *runtime constitutions* as a generalizable design pattern: a governance microkernel, a formal invariant engine with severity levels, a capability token system, and a canonical proof artifact constitute a constitutional framework that can be instantiated with different governance content for different domains. The execution gate, invariant engine, capability token verifier, and evidence bundle generator are architecturally separable from Project-AI's specific invariants, council configuration, and policy content.

16. CONCLUSION

We have presented the structural upgrade of Project-AI from a binary admissibility gate into a capability-scoped, selectively admissible, evidence-producing runtime governance architecture. The three primary contributions—Safe-Allow Calibration Layer, ConversationThreatStateRegister, and separated authorization pipeline—each address a distinct failure mode of the prior architecture without weakening its safety guarantees.

The governance liveness problem is addressed by distinguishing seven outcome semantics rather than two. The boiling-frog escalation problem is addressed by an eight-component stateful risk model that evaluates conversation trajectory rather than isolated turns. The approval reuse problem is addressed by requiring both type-level permission and instance-level authorization before a short-lived, scope-bound, replay-preventing capability token is minted.

Post-upgrade evaluation with the SemanticRiskClassifier is honest: benign allowed rate rises from 0% to 100% (+100pp, zero false positives, precision=1.000); harmful blocked rate is 67.7% (21/31)—the 10/31 false negatives use indirect framing that evades both embedding and lexical tiers. CTSR multi-turn detection improves from 1/5 to 4/5. The remaining gaps (10 FNs, 1 CTSR near-miss at conv_risk=0.429) are documented with concrete remediation steps in §14.

Every governed request produces a canonical EvidenceBundle containing full governance provenance. 186 tests across 19 test files validate all new components. Architecture documentation covers threat model, governance microkernel, governance contract, recovery protocol, and distributed governance roadmap.

Runtime constitutions—formally defined, capability-scoped, selectively admissible, evidence-producing—represent a tractable path toward AI governance properties that are verifiable, measurable, and honestly bounded rather than merely promised.

References.

- [1] Y. Bai et al., "Constitutional AI: Harmlessness from AI Feedback," arXiv:2212.08073, Anthropic, Dec. 2022.
- [2] P. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, "Deep Reinforcement Learning from Human Preferences," NeurIPS, 2017.
- [3] NIST, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," NIST AI 100-1, Jan. 2023.
- [4] European Parliament, "Artificial Intelligence Act," Regulation (EU) 2024/1689, Official Journal of the EU, Jul. 2024.

- [5] J. P. Anderson, "Computer Security Technology Planning Study," ESD-TR-73-51, Air Force Electronic Systems Division, Oct. 1972.
- [6] M. Leucker and C. Schallhart, "A Brief Account of Runtime Verification," *Journal of Logic and Algebraic Programming*, vol. 78, no. 5, 2009.
- [7] R. S. Sandhu and P. Samarati, "Access Control: Principles and Practice," *IEEE Communications Magazine*, vol. 32, no. 9, 1994.
- [8] J. Hu et al., "Attribute-Based Access Control," *IEEE Computer*, vol. 48, no. 2, 2015.
- [9] M. Sloman, "Policy Driven Management for Distributed Systems," *Journal of Network and Systems Management*, vol. 2, no. 4, 1994.
- [10] C. Wright, C. Morris, and S. Smalley, "Linux Security Modules: General Security Support for the Linux Kernel," *USENIX Security*, 2002.
- [11] The Linux Foundation, "eBPF: Extended Berkeley Packet Filter," ebpf.io, 2024.
- [12] NIST, "Zero Trust Architecture," NIST SP 800-207, Aug. 2020.
- [13] M. Miller, M. Samuel, B. Laurie, I. Awad, and M. Stay, "Caja: Safe Active Content in Sanitised JavaScript," Google, 2008.
- [14] L. Lamport, R. Shostak, and M. Pease, "The Byzantine Generals Problem," *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, 1982.
- [15] Z. Wang et al., "A Survey on Large Language Model-based Autonomous Agents," [arXiv:2308.11432](https://arxiv.org/abs/2308.11432), 2023.
- [16] P. Chao et al., "JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models," [arXiv:2404.01318](https://arxiv.org/abs/2404.01318), 2024.
- [17] L. Derczynski et al., "Garak: A Framework for Security Probing Large Language Models," [arXiv:2406.11036](https://arxiv.org/abs/2406.11036), 2024.
- [18] J. Karrick, "Constitutional Architectures for Sovereign AI Systems," Zenodo, DOI: 10.5281/zenodo.18794646, 2026.
- [19] J. Karrick, "OctoReflex Containment Model," Zenodo, DOI: 10.5281/zenodo.18726064, 2026.
- [20] J. Karrick, "TSCG-B Binary Wire Protocol Specification v1.0," Zenodo, DOI: 10.5281/zenodo.18826409, 2026.
- [21] J. Karrick, "T.A.M.S.-Omega: Meta-Constitutional Sovereignty Framework," Project-AI Repository, v1.0 Edition 1, Mar. 2026.
- [22] OWASP, "OWASP Top 10 for Large Language Model Applications," OWASP Foundation, v1.1, 2023.
- [23] IETF, "Internet X.509 PKI Time-Stamp Protocol (TSP)," RFC 3161, Aug. 2001.
- [24] J. Rushby, "Formal Methods and Their Role in the Certification of Critical Systems," SRI International Technical Report, 1995.
- [25] R. Perez and L. Ribeiro, "Ignore Previous Prompt: Attack Techniques for Language Models," *NeurIPS ML Safety Workshop*, 2022.
- [26] J. H. Saltzer and M. D. Schroeder, "The Protection of Information in Computer Systems," *Proceedings of the IEEE*, vol. 63, no. 9, 1975.
- [27] J. Karrick, "Project-AI Structural Upgrades: Capability-Scoped Execution Governance," Branch 05-09-26-Structural-upgrades, Project-AI Repository, Zenodo, DOI: 10.5281/zenodo.18794292, May 2026.